

Lecture 18:

Compiling packages

into Python

CEE 690

Write libraries in C++/C/Fortran and then compile as Python libraries

- **Execution speed:** C++ can be multiple orders of magnitude faster than native Python.
- **Memory management:** C++ gives you granular control over memory management.
- **True parallelism:** C++ can bypass the GIL so we can leverage OpenMP.
- **Access to legacy code:** Creating wrappers around legacy code allows us to access them from Python.
- **Encapsulation:** Able to control IP by providing .so files for Python instead of .py

There are a lot of ways of doing this

- **Pybind11**: Leverages C++ metaprogramming to handle types. Most common approach used nowadays.
- **Cython**: Write a separate file in a hybrid language that acts as a translator.
- **SWIG**: Interface generator that scans the code to figure and writes the wrappers for you
- **ctypes/cffi**: Load compiled C++ library directly into Python without changing anything.
- **F2py**: Scans fortran code to create the Python library

pybind11

pybind11: C++ code

```
#include <pybind11/pybind11.h>
#include <pybind11/numpy.h>

namespace py = pybind11;

double compute_mean(py::array_t<double> input_array) {
    // 1. Request a safe, 3-Dimensional proxy object from the array.
    // The <3> explicitly tells pybind11 we are expecting a 3D grid.
    auto grid = input_array.unchecked<3>();

    // Safety check
    if (input_array.size() == 0) {
        throw std::runtime_error("Cannot compute mean of an empty array.");
    }

    double sum = 0.0;

    // 2. Iterate using the proxy object's built-in shape() methods
    // NO pointers, NO pointer arithmetic, NO static casts.
    for (py::ssize_t i = 0; i < grid.shape(0); i++) {
        for (py::ssize_t j = 0; j < grid.shape(1); j++) {
            for (py::ssize_t k = 0; k < grid.shape(2); k++) {

                // 3. Access the data using standard mathematical coordinates
                sum += grid(i, j, k);
            }
        }
    }

    return sum / input_array.size();
}

// 4. The Python Binding
PYBIND11_MODULE(spatial_math, m) {
    m.doc() = "C++ Spatial Math Engine (Pointer-Free)";
    m.def("compute_mean", &compute_mean, "Compute the spatial mean of a 3D NumPy array");
}
```

pybind11: Compile

- Conda/Pip install pybind11

```
# --- SETTINGS ---
CXX      = g++
# Optimized for speed (-O3) and required for shared libs (-fPIC)
CXXFLAGS = -O3 -Wall -shared -std=c++17 -fPIC

# --- PYTHON CONFIG ---
# This command automatically finds the correct include paths for pybind11
PY_INCLUDES = $(shell python3 -m pybind11 --includes)
# This handles the suffix (e.g., .cpython-310-x86_64-linux-gnu.so)
PY_SUFFIX   = $(shell python3-config --extension-suffix)

# --- TARGETS ---
TARGET      = spatial_math$(PY_SUFFIX)
SRC         = spatial_mean.cpp

# --- RULES ---
all: $(TARGET)

$(TARGET): $(SRC)
    $(CXX) $(CXXFLAGS) $(PY_INCLUDES) $(SRC) -o $(TARGET)

clean:
    rm -f *$(PY_SUFFIX)
```

```
(cee690) nc153 $ make
[g++ -O3 -Wall -shared -std=c++17 -fPIC -I/hpc/home/nc153/miniconda3/envs/cee690/include/python3.14 -I/hpc/home/nc153/miniconda3/envs/cee690/lib/python3.14/site-packages/pybind11/include spatial_mean.cpp -o spatial_math.cpython-314-x86_64-linux-gnu.so]
```


pybind11 module: Run from Python

```
import numpy as np
import spatial_math
import time

def run_benchmark():
    # 1. Setup dimensions (e.g., 100 days, 180 latitudes, 360 longitudes)
    # Total points: ~6.4 million
    shape = (100, 180, 360)

    # IMPORTANT: Use float64 to match 'double' in C++
    data = np.random.rand(*shape).astype(np.float64)

    # 2. Test the C++ Extension
    print("Invoking C++ Engine...")
    start_cpp = time.time()
    cpp_result = spatial_math.compute_mean(data)
    end_cpp = time.time()

    # 3. Test Native NumPy (for verification)
    print("Invoking NumPy (Reference)...")
    start_np = time.time()
    np_result = np.mean(data)
    end_np = time.time()

    # 4. Report Results
    print("-" * 35)
    print(f"C++ Result:      {cpp_result:.8f}")
    print(f"NumPy Result:    {np_result:.8f}")
    print(f"Difference:      {abs(cpp_result - np_result):.2e}")
    print("-" * 35)
    print(f"C++ Time:        {end_cpp - start_cpp:.6f} seconds")
    print(f"NumPy Time:      {end_np - start_np:.6f} seconds")

    if np.allclose(cpp_result, np_result):
        print("\nSUCCESS: The C++ wrapper is mathematically accurate!")
    else:
        print("\nERROR: Mismatch detected. Check data types or indexing.")

if __name__ == "__main__":
    run_benchmark()
```

```
[(cee690) nc153 $ python test_spatial.py
Invoking C++ Engine...
Invoking NumPy (Reference)...
-----
C++ Result:      0.50000614
NumPy Result:    0.50000614
Difference:      5.34e-14
-----
C++ Time:        0.004930 seconds
NumPy Time:      0.003759 seconds
```



Cython: Write in hybrid form

```
# cython: boundscheck=False
# cython: wraparound=False

import numpy as np
cimport numpy as cnp

def compute_mean(double[:, :, ::1] grid):
    """Compute the spatial mean of a 3D NumPy array using Cython."""

    # 1. Statically type the loop variables
    cdef Py_ssize_t i, j, k
    cdef Py_ssize_t x_max = grid.shape[0]
    cdef Py_ssize_t y_max = grid.shape[1]
    cdef Py_ssize_t z_max = grid.shape[2]

    cdef double total_sum = 0.0
    cdef double total_elements = x_max * y_max * z_max

    # 2. The Heavy Lifting
    for i in range(x_max):
        for j in range(y_max):
            for k in range(z_max):
                # 3. Direct memory access
                total_sum += grid[i, j, k]

    return total_sum / total_elements
```


Cython: Compile

```
from setuptools import setup
from Cython.Build import cythonize
import numpy as np

setup(
    name="Spatial Math Cython",
    # cythonize() translates the .pyx to .c and compiles it
    ext_modules=cythonize(
        "spatial_mean_cython.pyx",
        compiler_directives={'language_level': "3"}
    ),
    # Ensure the C compiler can find the NumPy headers
    include_dirs=[np.get_include()]
)
```

```
[(cee690) nc153 $ python3 setup.py build_ext --inplace
Compiling spatial_mean_cython.pyx because it changed.
[[1/1] Cythonizing spatial_mean_cython.pyx
[running build_ext
building 'spatial_mean_cython' extension
creating build/temp.linux-x86_64-cpython-314
gcc -pthread -B /hpc/home/nc153/miniconda3/envs/cee690/share/python_compiler_compat -fno-strict-overflow -Wsign-compare -D
NDEBUG -O2 -Wall -fPIC -O2 -isystem /hpc/home/nc153/miniconda3/envs/cee690/include -fPIC -O2 -isystem /hpc/home/nc153/min
iconda3/envs/cee690/include -fPIC -I/hpc/home/nc153/miniconda3/envs/cee690/lib/python3.14/site-packages/numpy/_core/include
-I/hpc/home/nc153/miniconda3/envs/cee690/include/python3.14 -c spatial_mean_cython.c -o build/temp.linux-x86_64-cpython-3
14/spatial_mean_cython.o
creating build/lib.linux-x86_64-cpython-314
gcc -pthread -B /hpc/home/nc153/miniconda3/envs/cee690/share/python_compiler_compat -shared -Wl,--allow-shlib-undefined -Wl
l,-rpath,/hpc/home/nc153/miniconda3/envs/cee690/lib -Wl,-rpath-link,/hpc/home/nc153/miniconda3/envs/cee690/lib -L/hpc/home
/nc153/miniconda3/envs/cee690/lib -Wl,--allow-shlib-undefined -Wl,-rpath,/hpc/home/nc153/miniconda3/envs/cee690/lib -Wl,-r
path-link,/hpc/home/nc153/miniconda3/envs/cee690/lib -L/hpc/home/nc153/miniconda3/envs/cee690/lib build/temp.linux-x86_64-
cpython-314/spatial_mean_cython.o -o build/lib.linux-x86_64-cpython-314/spatial_mean_cython.cpython-314-x86_64-linux-gnu.s
o
copying build/lib.linux-x86_64-cpython-314/spatial_mean_cython.cpython-314-x86_64-linux-gnu.so ->
```


Cython module: Run from Python

```
from setuptools import setup
from Cython.Build import cythonize
import numpy as np

setup(
    name="Spatial Math Cython",
    # cythonize() translates the .pyx to .c and compiles it
    ext_modules=cythonize(
        "spatial_mean_cython.pyx",
        compiler_directives={'language_level': "3"}
    ),
    # Ensure the C compiler can find the NumPy headers
    include_dirs=[np.get_include()]
)
```

```
[(cee690) nc153 $ python3 setup.py build_ext --inplace
Compiling spatial_mean_cython.pyx because it changed.
[[1/1] Cythonizing spatial_mean_cython.pyx
[running build_ext
building 'spatial_mean_cython' extension
creating build/temp.linux-x86_64-cpython-314
gcc -pthread -B /hpc/home/nc153/miniconda3/envs/cee690/share/python_compiler_compat -fno-strict-overflow -Wsign-compare -D
NDEBUG -O2 -Wall -fPIC -O2 -isystem /hpc/home/nc153/miniconda3/envs/cee690/include -fPIC -O2 -isystem /hpc/home/nc153/mini
conda3/envs/cee690/include -fPIC -I/hpc/home/nc153/miniconda3/envs/cee690/lib/python3.14/site-packages/numpy/_core/include
-I/hpc/home/nc153/miniconda3/envs/cee690/include/python3.14 -c spatial_mean_cython.c -o build/temp.linux-x86_64-cpython-3
14/spatial_mean_cython.o
creating build/lib.linux-x86_64-cpython-314
gcc -pthread -B /hpc/home/nc153/miniconda3/envs/cee690/share/python_compiler_compat -shared -Wl,--allow-shlib-undefined -Wl
l,-rpath,/hpc/home/nc153/miniconda3/envs/cee690/lib -Wl,-rpath-link,/hpc/home/nc153/miniconda3/envs/cee690/lib -L/hpc/home
/nc153/miniconda3/envs/cee690/lib -Wl,--allow-shlib-undefined -Wl,-rpath,/hpc/home/nc153/miniconda3/envs/cee690/lib -Wl,-r
path-link,/hpc/home/nc153/miniconda3/envs/cee690/lib -L/hpc/home/nc153/miniconda3/envs/cee690/lib build/temp.linux-x86_64-
cpython-314/spatial_mean_cython.o -o build/lib.linux-x86_64-cpython-314/spatial_mean_cython.cpython-314-x86_64-linux-gnu.s
o
copying build/lib.linux-x86_64-cpython-314/spatial_mean_cython.cpython-314-x86_64-linux-gnu.so ->
```

ctypes



ctypes: C code

```
// Notice: ZERO Python or pybind11 includes!

extern "C" {
    // We pass a flat double pointer and the total size of the grid
    double compute_mean(const double* array, long total_elements) {
        if (total_elements <= 0) return 0.0;

        double sum = 0.0;

        // Iterate over the flat memory block
        for (long i = 0; i < total_elements; i++) {
            sum += array[i];
        }

        return sum / total_elements;
    }
}
```


ctypes: Compile

```
CXX      = g++
# -O3: Maximum speed optimization for the math loops
# -Wall: Show all warnings
# -shared -fPIC: Mandatory flags for building a .so dynamic library
CXXFLAGS = -O3 -Wall -shared -fPIC

# --- FILES ---
TARGET    = libspatial.so
SRC       = spatial_mean_ctypes.cpp

# --- RULES ---

# Default target when the user types 'make'
all: $(TARGET)

# Compile the C++ shared library
$(TARGET): $(SRC)
    $(CXX) $(CXXFLAGS) $(SRC) -o $(TARGET)
    @echo "SUCCESS: Compiled $(TARGET)"

# Clean up the build files
clean:
    rm -f $(TARGET)
    @echo "Cleaned up $(TARGET)"

~
```

```
[(cee690) nc153 $ make
g++ -O3 -Wall -shared -fPIC spatial_mean_ctypes.cpp -o libspatial.so
```

ctypes: Run from Python

```
import numpy as np
import ctypes
import numpy.ctypeslib as npct
import time
import os

def run_ctypes_benchmark():
    # 1. Load the shared library
    # We use os.path.abspath to ensure Python finds the .so file in the current dir
    lib_path = os.path.abspath('./libspatial.so')
    spatial_lib = ctypes.CDLL(lib_path)

    # 2. Define the NumPy C-Pointer Type
    # We strictly enforce that the array MUST be double (float64),
    # MUST be 3D, and MUST be contiguous in memory.
    array_3d_double = npct.ndpointer(dtype=np.float64, ndim=3, flags='CONTIGUOUS')

    # 3. Define the Function Signature (CRITICAL STEP)
    # void compute_mean(double* array, long total_elements)
    spatial_lib.compute_mean.argtypes = [array_3d_double, ctypes.c_long]
    spatial_lib.compute_mean.restype = ctypes.c_double

    # 4. Generate the Climate Grid
    shape = (100, 180, 360)
    print(f"Generating synthetic grid of shape {shape}...")
    data = np.random.rand(*shape).astype(np.float64)

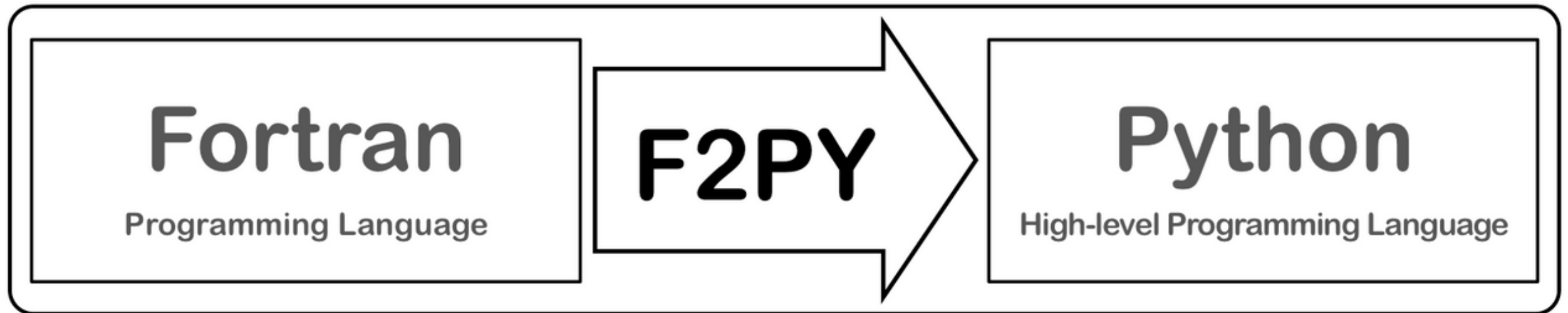
    # 5. Execute the C++ function
    print("Invoking ctypes C++ Engine...")
    start = time.time()
    # Notice we pass data.size to give the C++ code the total element count
    cpp_result = spatial_lib.compute_mean(data, data.size)
    end = time.time()

    # 6. Verify against NumPy
    np_result = np.mean(data)

    print("-" * 35)
    print(f"C++ (ctypes) Result: {cpp_result:.8f}")
    print(f"NumPy Result: {np_result:.8f}")
    print(f"Execution Time: {end - start:.4f} seconds")
    print("-" * 35)
```

```
(cee690) nc153 $ python test_ctypes.py
Generating synthetic grid of shape (100, 180, 360)...
Invoking ctypes C++ Engine...
-----
C++ (ctypes) Result: 0.50013798
NumPy Result: 0.50013798
Execution Time: 0.0048 seconds
-----
```


F2PY



f2py: Fortran code

```
subroutine compute_mean(grid, nx, ny, nz, mean_val)
  implicit none

  ! 1. Variable Declarations
  integer, intent(in) :: nx, ny, nz
  real(8), intent(in) :: grid(nx, ny, nz)
  real(8), intent(out) :: mean_val

  ! 2. f2py Directives (The "Magic")
  ! Tell f2py that Python only needs to pass 'grid'.
  ! It will automatically calculate nx, ny, nz using NumPy's shape() function
  !f2py intent(in) :: grid
  !f2py intent(hide), depend(grid) :: nx = shape(grid,0), ny = shape(grid,1), nz = shape(grid,2)
  !f2py intent(out) :: mean_val

  ! Local variables
  integer :: i, j, k
  real(8) :: total_sum, total_elements

  total_sum = 0.0d0
  total_elements = dble(nx) * dble(ny) * dble(nz)

  if (total_elements == 0.0d0) then
    mean_val = 0.0d0
    return
  end if

  ! 3. The Math Loop
  ! CRITICAL: Fortran is Column-Major! The innermost loop MUST be the first
  do k = 1, nz
    do j = 1, ny
      do i = 1, nx
        total_sum = total_sum + grid(i, j, k)
      end do
    end do
  end do

  mean_val = total_sum / total_elements

end subroutine compute_mean
```

f2py: Compile

```
# --- CONFIGURATION ---
PYTHON = python3
F2PY    = $(PYTHON) -m numpy.f2py

# --- FILES ---
MODULE_NAME = spatial_mean_f2py
SRC          = spatial_mean.f90

# --- RULES ---

# Default target
all: build

# Compile the Fortran code into a Python extension
build: $(SRC)
    @echo "Compiling Fortran to Python using f2py..."
    $(F2PY) -c $(SRC) -m $(MODULE_NAME)
    @echo "SUCCESS: Compiled $(MODULE_NAME) module"

# Clean up the build files and generate
clean:
    @echo "Cleaning up generated library"
    rm -f $(MODULE_NAME).so
    rm -rf .f2py_f2cmap
    rm -rf __pycache__
```

```
(cee690) nc153 $ make
Compiling Fortran to Python using f2py...
python3 -m numpy.f2py -c spatial_mean.f90 -m spatial_mean_f2py
Cannot use distutils backend with Python>=3.12, using meson backend instead.
Using meson backend
Will pass --lower to f2py
See https://numpy.org/doc/stable/f2py/buildtools/meson.html
Reading fortran codes...
  Reading file 'spatial_mean.f90' (format:free)
Line #13 in spatial_mean.f90:"          intent(hide), depend(grid) :: nx = sha
  analyzeline: cannot handle multiple attributes without type specificat
Post-processing...
  Block: spatial_mean_f2py
    Block: compute_mean
Applying post-processing hooks...
  character_backward_compatibility_hook
```


F2py: Run from Python

```
import numpy as np
import spatial_mean_f2py
import time

def run_fortran_benchmark():
    shape = (100, 180, 360)
    print(f"Generating synthetic grid of shape {shape}...")

    # HPC PRO-TIP: order='F' ensures the memory is laid out correctly
    # Without this, f2py will secretly copy the entire array!
    data = np.random.rand(*shape).astype(np.float64, order='F')

    print("Invoking Fortran Engine...")
    start = time.time()

    # Notice how clean the call is! We don't pass nx, ny, or nz.
    # The !f2py intent(hide) directives handled it.
    fortran_result = spatial_mean_f2py.compute_mean(data)

    end = time.time()

    np_result = np.mean(data)

    print("-" * 35)
    print(f"Fortran Result: {fortran_result:.8f}")
    print(f"NumPy Result: {np_result:.8f}")
    print(f"Execution Time: {end - start:.4f}")
    print("-" * 35)

if __name__ == "__main__":
    run_fortran_benchmark()
```

```
(cee690) nc153 $ python test_f2py.py
Generating synthetic grid of shape (100, 180, 360)...
Invoking Fortran Engine...
-----
Fortran Result: 0.49995773
NumPy Result: 0.49995773
Execution Time: 0.0047 seconds
-----
```


**Using pybind11 to create a
netCDF wrapper**

Wrapper code in C++

```
#include <pybind11/pybind11.h>
#include <pybind11/numpy.h>
#include <netcdf>
#include <ncException.h> // <-- Added modern exception header
#include <string>
#include <vector>
#include <stdexcept>

namespace py = pybind11;
using namespace netCDF;

// The function: Takes a filename and a variable name, returns a NumPy array
py::array_t<double> read_netcdf_2d(const std::string& filename, const std::string& var_name) {
    try {
        // 1. Open the file in read-only mode using the C++ API
        NcFile dataFile(filename, NcFile::read);

        // 2. Get the variable object
        NcVar var = dataFile.getVar(var_name);
        if (var.isNull()) {
            throw std::runtime_error("Variable '" + var_name + "' not found in file.");
        }

        // 3. Check dimensions (Ensure it's actually 2D)
        std::vector<NcDim> dims = var.getDims();
        if (dims.size() != 2) {
            throw std::runtime_error("This specific wrapper only handles 2D variables.");
        }
    }
}
```

Wrapper code in C++

```
#include <pybind11/pybind11.h>
#include <pybind11/numpy.h>
#include <netcdf>
#include <ncException.h> // <-- Added modern exception header
#include <string>
#include <vector>
#include <stdexcept>

namespace py = pybind11;
using namespace netCDF;

// The function: Takes a filename and a variable name, returns a NumPy array
py::array_t<double> read_netcdf_2d(const std::string& filename, const std::string& var_name) {
    try {
        // 1. Open the
        NcFile dataFile

        // 2. Get the v
        NcVar var = dat
        if (var.isNull()
            throw std::

        // 3. Check dim
        std::vector<NcD
        if (dims.size()
            throw std::

        size_t rows = dims[0].getSize();
        size_t cols = dims[1].getSize();

        // 4. Allocate the NumPy array in memory
        // We tell pybind11 the shape we need based on the NetCDF metadata
        py::array_t<double> result_array({rows, cols});

        // 5. THE MAGIC STEP: Direct Memory Read
        // We get a mutable pointer to the NumPy array's raw memory buffer
        // and tell the NetCDF library to read the disk data directly into it!
        var.getVar(result_array.mutable_data());

        return result_array;

    } catch (netCDF::exceptions::NcException& e) { // <-- Updated Conda namespace
        // Catch NetCDF specific C++ errors and pass them to Python
        throw std::runtime_error(std::string("NetCDF Error: ") + e.what());
    }
}

// 6. The pybind11 Module Definition
PYBIND11_MODULE(fast_nc, m) {
    m.doc() = "High-speed C++ NetCDF reader for Python";
    m.def("read_2d", &read_netcdf_2d, "Read a 2D double-precision variable directly into NumPy");
}
```


Makefile

```
CXX      = g++
CXXFLAGS = -O3 -Wall -shared -std=c++17 -fPIC

# Python & pybind11 dynamic paths
PY_INCLUDES = $(shell python3 -m pybind11 --includes)
PY_SUFFIX   = $(shell python3-config --extension-suffix)

# Our custom HPC User-Space paths
MY_PREFIX   = /hpc/home/nc153/miniconda3/envs/cee690
NC_INCLUDES = -I$(MY_PREFIX)/include
NC_LDFLAGS   = -L$(MY_PREFIX)/lib -L$(MY_PREFIX)/lib64
NC_LIBS      = -lnetcdf-cxx4 -lnetcdf

# The RPATH burns the library location into the .so file
# so Python can find libnetcdf.so at runtime without LD_LIBRARY_PATH!
RPATH       = -Wl,-rpath,$(MY_PREFIX)/lib:$(MY_PREFIX)/lib64

# Targets
TARGET      = fast_nc$(PY_SUFFIX)
SRC         = nc_reader.cpp

all: $(TARGET)

$(TARGET): $(SRC)
    $(CXX) $(CXXFLAGS) $(PY_INCLUDES) $(NC_INCLUDES) \
    $(SRC) -o $(TARGET) \
    $(NC_LDFLAGS) $(NC_LIBS) $(RPATH)
    @echo "SUCCESS: Built $(TARGET)"

clean:
    rm -f *$(PY_SUFFIX)
    @echo "Cleaned directory."
```

Run from python

```
import numpy as np
import netCDF4 as nc
import fast_nc
import time

def create_dummy_data(filename):
    print(f"Creating dummy NetCDF file: {filename}...")
    # Create a 5,000 x 5,000 grid (25 million points)
    shape = (5000, 5000)

    with nc.Dataset(filename, 'w', format='NETCDF4') as ds:
        ds.createDimension('x', shape[0])
        ds.createDimension('y', shape[1])

        # Create a double-precision (f8) variable
        temp_var = ds.createVariable('temperature', 'f8', ('x', 'y'))

        # Fill it with random data
        temp_var[:] = np.random.rand(*shape)
    print("File created successfully.\n")

def test_custom_reader(filename):
    print("Reading data using custom C++ engine...")

    start = time.time()
    # Call our pybind11 wrapper!
    data = fast_nc.read_2d(filename, "temperature")
    end = time.time()

    print("-" * 35)
    print(f"Array Shape: {data.shape}")
    print(f>Data Type: {data.dtype}")
    print(f"Mean Value: {np.mean(data):.6f}")
    print(f"Read Time: {end - start:.4f} seconds")
    print("-" * 35)

if __name__ == "__main__":
    test_file = "test_climate_data.nc"
    create_dummy_data(test_file)
    test_custom_reader(test_file)
```

```
(cee690) nc153 $ python test_reader.py
Creating dummy NetCDF file: test_climate_data.nc...
File created successfully.
```

```
Reading data using custom C++ engine...
```

```
-----
Array Shape: (5000, 5000)
Data Type: float64
Mean Value: 0.499980
Read Time: 0.2717 seconds
-----
```

Take home message

- Yes, you can just use Numba for the vast majority of what we just talked about.
- However, Numba will only work for basic compiled code (e.g., nested for loops)
- If you want to create wrappers around existing libraries, you will have to use another tool (e.g., pybind11, f2py...)